M.Sc. Engg. (CSE) Thesis

Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models

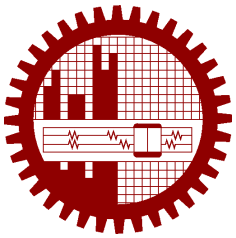
Submitted by

Md. Sakif Khan

0421052099

Supervised by

Dr. Sadia Sharmin



Submitted to

Department of Computer Science and Engineering

Bangladesh University of Engineering and Technology

Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of

Master of Science in Computer Science and Engineering

June 2026

Candidate's Declaration

I, do, hereby, certify that the work presented in this thesis, titled, "Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models", is the outcome of the investigation and research carried out by me under the supervision of Dr. Sadia Sharmin, Associate Professor, Department of CSE, BUET.

I also declare that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Md. Sakif Khan

0421052099

The thesis titled "**Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models**", submitted by Md. Sakif Khan, Student ID 0421052099, Session April 2021, to the Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, has been accepted as satisfactory in partial fulfilment of the requirements for the degree of Master of Science in Computer Science and Engineering and approved as to its style and contents on June 6, 2026.

Board of Examiners

1. _____

   Dr. Sadia Sharmin — Chairman
   Associate Professor — (Supervisor)
   Department of CSE, BUET, Dhaka

2. _____

   Dr. Tanzima Hashem — Member
   Professor and Head — (Ex-Officio)
   Department of CSE, BUET, Dhaka

3. _____

   Dr. M. Kaykobad — Member
   Professor
   Department of CSE, BUET, Dhaka

4. _____

   Dr. Abu Sayed Md. Latiful Hoque — Member
   Professor
   Department of CSE, BUET, Dhaka

5. _____

   Dr. Honorable External Examiner — Member
   Professor — (External)
   Department of CSE
   University of External, Dhaka

Acknowledgement

I am thankful to

Finally,

Dhaka
June 6, 2026

Md. Sakif Khan
0421052099

Contents

List of Figures

List of Tables

List of Algorithms

Abstract

Write your thesis abstract here.

These are some dummy text used as page fillers only.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras et ultricies massa. Nulla a sapien lobortis, dignissim nibh in, aliquet mauris. Integer at dictum metus. Quisque in tortor congue ipsum ultricies tristique. Maecenas ut tortor dapibus, sagittis enim at, tincidunt massa. Ut sollicitudin sagittis ipsum, ac tincidunt quam gravida ac. Nullam quis faucibus purus. Aliquam vel pretium turpis. Aliquam a quam non ex interdum sagittis id vitae quam. Nullam sodales ligula malesuada maximus consequat. Proin a justo eget lacus vulputate maximus luctus vitae enim. Aliquam libero turpis, pharetra a tincidunt ac, pulvinar sit amet urna. Pellentesque eget rutrum diam, in faucibus sapien. Aenean sit amet est felis. Aliquam dolor eros, porttitor quis volutpat eget, posuere a ligula. Proin id velit ac lorem finibus pellentesque.

Maecenas vitae interdum mi. Aenean commodo nisl massa, at pharetra libero cursus vitae. In hac habitasse platea dictumst. Suspendisse iaculis euismod dui, et cursus diam. Nullam euismod, est ut dapibus condimentum, lorem eros suscipit risus, sit amet hendrerit justo tortor nec lorem. Morbi et mi eget erat bibendum porta. Ut tristique ultricies commodo. Nullam iaculis ligula sed lacinia ornare.

Sed ultricies cursus nisi at vestibulum. Aenean laoreet viverra efficitur. Ut eget sapien lorem. Mauris malesuada, augue in pulvinar consectetur, ex tortor tristique ligula, sit amet faucibus metus lectus interdum nisl. Nam eget turpis vitae ligula pulvinar bibendum a ut ipsum. Mauris fringilla lacinia malesuada. Fusce id orci velit. Donec tristique rhoncus urna, a hendrerit arcu vehicula imperdiet. Integer tristique erat at gravida condimentum. Sed ornare cursus quam, eget tincidunt enim bibendum sed. Aliquam elementum ligula scelerisque leo sagittis, quis convallis elit dictum. Donec sit amet orci aliquam, ultricies sapien nec, gravida nisi. Etiam et pulvinar diam, et pellentesque arcu. Nulla interdum metus sed aliquet consequat.

Proin in mi id nulla interdum aliquet ac quis arcu. Duis blandit sapien commodo turpis hendrerit pharetra. Phasellus sit amet justo orci. Proin mattis nisl dictum viverra fringilla. Interdum et malesuada fames ac ante ipsum primis in faucibus. Curabitur facilisis euismod augue vestibulum tincidunt. Nullam nulla quam, volutpat vitae efficitur eget, porta sit amet nunc. Phasellus pharetra est eget urna ornare volutpat. Aenean ultrices, libero eget porttitor fringilla, purus tortor accumsan neque, sit amet viverra felis tortor eget justo. Nunc id metus a purus tempus euismod condimentum non lacus. Nam vitae diam aliquam, facilisis diam quis, pharetra nunc. Nulla eget vestibulum tellus, ut cursus tellus. Vestibulum euismod pellentesque sodales.

Maecenas at mi interdum, faucibus lorem sed, hendrerit nisi. In vitae augue consequat diam commodo porta sit amet eu purus. Mauris mattis condimentum feugiat. Nulla commodo molestie risus vitae maximus. Proin hendrerit neque malesuada urna laoreet convallis. Etiam a diam pulvinar, auctor sem ac, hendrerit risus. Ut urna urna, venenatis ac tellus non, scelerisque tristique ligula. Vestibulum sollicitudin vel leo malesuada accumsan. Donec sit amet erat diam. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Vivamus odio dui, scelerisque et lorem egestas, posuere ullamcorper nunc. Integer varius nunc nec velit tincidunt commodo. Mauris rhoncus ultrices sapien non suscipit.

End of dummy text.

Chapter 1

Introduction

Large language models answer factual questions fluently and, often enough, incorrectly. The failure is not random noise: it is systematic, confident, and hard to detect from the output alone, because a fabricated answer is written in the same register as a correct one. Where the answer requires composing several facts rather than recalling one, the failure rate rises sharply. This thesis is about closing that gap by giving a language model a symbolic knowledge graph to walk through, and by checking what it is about to assert before it asserts it.

This chapter motivates the problem, surveys briefly why the obvious remedies are incomplete, states the problem and research questions the thesis addresses, summarises the contributions, and sets out the organisation of the remainder of the document.

1.1 Hallucination in Large Language Models

A *hallucination* is a generated statement that is fluent, syntactically well formed, and not supported by any source the model can point to — frequently one that is simply false. Huang et al. [1] survey the phenomenon at length and treat it as an intrinsic consequence of how these models are built rather than an incidental defect to be patched away. That framing matters for this thesis: if hallucination were a bug, the response would be to fix it; because it is structural, the response must be architectural.

1.1.1 Where Hallucination Originates: Training Data, Model, and Prompt

It is useful to separate three loci at which a hallucination can originate, because they admit very different interventions.

The first is the *training data*. A model trained on a corpus containing errors, staleness, or under-representation of a domain will reproduce those defects. Correcting them means curating the corpus and retraining, which is outside the reach of anyone who is not the model's producer.

The second is the *model* itself — its parameterisation and decoding procedure. Knowledge is stored diffusely across weights with no explicit index, so the model has no reliable internal signal distinguishing a fact it has memorised from a plausible continuation it is constructing. Interventions here mean changing the architecture or the training objective, again largely out of reach.

The third is the *prompt*, and this is the locus this thesis addresses. If the context supplied alongside the question contains the facts required to answer it, the model need not fall back on parametric recall. Structured context can override defects inherited from training data without touching the model. This is the premise behind retrieval augmentation [2], and it is the premise behind the present work; the contribution lies in *how* that context is assembled and in *what is checked* before the answer is emitted.

1.1.2 Why Multi-Hop Questions Are the Hard Case

Single-fact questions are comparatively benign. "Where was Ada Lovelace born?" requires one lookup, and either the model knows it or it does not.

Multi-hop questions — "Who directed the film that won Best Picture in 1998?" — require composing an ordered chain of lookups, each conditioned on the previous result. Two properties make these the hard case. First, errors compound: a wrong resolution at the first hop is not usually detected downstream, and the remainder of the chain proceeds confidently from a false premise. Second, the intermediate results are exactly what a question-level correctness check cannot see. A system that answers correctly by accident and a system that answers correctly by sound reasoning are indistinguishable from the final string alone.

Multi-hop questions also frequently carry *constraints* — temporal, categorical, or ordinal — that must be satisfied jointly rather than sequentially, and dropping a constraint silently is among the most common ways such questions are answered wrongly rather than not at all.

1.2 From Retrieval Augmentation to Graph-Structured Grounding

1.2.1 Vector Retrieval and Its Structural Blind Spot

Retrieval-Augmented Generation [2] attaches the model to an external corpus: the question is embedded, semantically similar passages are retrieved by vector similarity, and the model answers from them. Where the answer resides in a single retrievable passage, this works well.

Its weakness is structural. Vector retrieval treats a corpus as flat chunks of text and ranks them by similarity to the question. For a multi-hop question the similarity signal is misleading, because

the passage containing the *intermediate* entity may bear little lexical or semantic resemblance to the original question — the system does not yet know which entity to look for. Relationships between entities, which are precisely what a compositional question traverses, are not represented at all; they are at best implicit in adjacency within a retrieved chunk. Surveys of the intersection between graphs and language models identify this as the central limitation of flat retrieval [3].

1.2.2 Knowledge Graphs and Static GraphRAG

A *knowledge graph* makes relationships explicit. Facts are stored as typed edges between identified entities, so composing facts becomes traversal rather than inference over prose. GraphRAG [4] exploits this by building a graph over a corpus and retrieving structured context rather than passages, and related systems extend the idea to long-term memory [5] and to temporally-qualified facts [6].

These systems nonetheless retrieve *statically*: a subgraph around the linked entities is fetched in one shot and handed to the model. The retrieval step does not know what the reasoning step will need, because it happens first and only once. For a question whose second hop depends on the answer to its first, a fixed-radius neighbourhood is either too small to contain the answer or so large that the relevant edges are buried among thousands of irrelevant ones.

1.2.3 Agentic Retrieval: Reasoning as Navigation

The natural response is to interleave retrieval with reasoning: let the model take one step, observe what it found, and decide where to look next. Retrieval becomes *navigation*, and the system becomes an agent operating on the graph rather than a pipeline reading from it. Think-on-Graph [7] established the pattern with LLM-guided beam search over the graph; Plan-on-Graph [8] added sub-objective decomposition and reflection-driven backtracking; Reasoning-on-Graphs [9] generates grounded relation paths as explicit plans; and further systems add tool abstractions [10], handle incomplete graphs [11], or generalise across structured sources [12]. Recent surveys treat this convergence of language models and knowledge graphs as a distinct research programme [13, 14].

Agentic navigation is the right shape for the problem, and this thesis adopts it. The question is what remains unsolved once it is adopted.

1.3 Limitations of Existing Agentic KGQA Systems

1.3.1 No Check on What the Final Answer Asserts

Agentic systems ground their answers in a traversed subgraph, and this thesis's own measurements confirm that they do so effectively: entities they assert do exist in the graph, and are reachable along the paths they walked (Section 8.5). The deficit is not fabrication.

The deficit is *precision*. Every system surveyed in Chapter 3 emits whatever its final generation call produces from the traversed context, with no check that each asserted entity actually answers the question that was asked. An entity can be present in the retrieved subgraph, correctly retrieved, and still be the wrong answer — an intermediate node from the reasoning chain, a container when a member was requested, or one of several plausible neighbours selected arbitrarily. It can also be one of many entities asserted where the question admits few, which inflates recall at the cost of precision. Both failures are invisible to a system that verifies retrieval but never verifies assertion; they surface as a measurable precision gap against a system that does check (Section 8.2).

Chain-of-Verification [15] and multi-agent debate [16] establish that post-hoc self-checking reduces factual error in free-form generation, and a self-correcting agentic graph pipeline has been demonstrated in a clinical setting [17]. What is missing is a claim-level check against the retrieved triples performed *before* the answer is emitted, inside a graph-navigating agent.

1.3.2 Unquantified Contribution of Individual Agentic Mechanisms

Agentic systems are assemblies of mechanisms — decomposition, scoring, backtracking, self-correction — and are reported as wholes. When such a system outperforms a static baseline, the literature rarely establishes which mechanism produced the gain, or whether some mechanisms contribute nothing. Without that attribution, subsequent work inherits the entire assembly on faith and its cost along with it. Component-level ablation is the standard remedy and is conspicuously underused in this specific literature.

1.3.3 Accuracy Reported Without Cost

Navigation is expensive. Each exploration step costs at least one model call, and agentic systems routinely consume an order of magnitude more tokens per question than a single-shot baseline. Accuracy tables that omit this make agentic methods look uniformly preferable when the honest picture is a trade-off, and the trade-off is exactly what a practitioner deciding whether to deploy such a system needs to see.

1.4 Problem Statement

This thesis addresses the following problem. Given a natural-language question q and a knowledge graph $G = (E, R, T)$ with entities E, relation types R, and triples $T \subseteq E \times R \times E$, produce an answer set $A \subseteq E$ together with a supporting triple set $S \subseteq T$, such that every entity in A is justified by S, and S consists only of triples the system actually traversed in reaching A.

Three properties distinguish this from standard knowledge graph question answering. The output is a *pair*, not an answer alone: the supporting triples are part of the contract, which makes the reasoning auditable rather than asserted. The support must be *traversed*, not merely retrievable after the fact, which forecloses post-hoc justification of an answer arrived at some other way. And the justification is checked *before* the answer is emitted, so that unsupported content can be removed or hedged rather than merely flagged.

1.5 Research Questions

1.5.1 RQ1: Does Agentic Navigation Improve Multi-Hop Factual Accuracy?

Does interleaving retrieval with reasoning over a knowledge graph produce more accurate answers than static retrieval, and does the advantage grow with the number of hops a question requires? Answering this needs a no-retrieval control, since the standard benchmarks predate current models and may be partly memorised.

1.5.2 RQ2: What Does Pre-Generation Verification Contribute Beyond Graph Navigation?

Given a system that already navigates the graph, what does a claim-level check against traversed triples add? The question is posed as *what does it contribute*, not *does it reduce hallucination*, because the answer turns out to have several parts that a yes-or-no framing would obscure (Section 8.8).

1.5.3 RQ3: Which Components Contribute What, at What Token Cost?

Decomposition, backtracking, verification, and learned candidate scoring each cost model calls. Which of them measurably improve accuracy or groundedness, by how much, and at what price in tokens and latency?

1.6 Our Contribution

1.6.1 The AGR Framework and Its Verification Layer

We present Agentic Graph Reasoning (AGR), a knowledge-graph question-answering framework built as an explicit state machine over a constrained, deterministic graph tool API, and carrying a *Structural Verification Layer* that decomposes the draft answer into atomic claims and checks each against the traversed subgraph before the answer is emitted. Unsupported claims trigger either targeted re-exploration or removal from the answer. The system returns the answer paired with its supporting triples.

1.6.2 A Component-Level Ablation of Agentic Mechanisms

We ablate the planner, the backtracking mechanism, the verification layer, and the learned component of candidate scoring, reporting accuracy, groundedness, and token cost for each condition with paired significance testing. This addresses the attribution gap identified in Section 1.3.

1.6.3 Stratum-Dependent Decomposition: When Planning Hurts

Question decomposition is treated in the literature as straightforwardly beneficial. We find it is not: removing the planner *improves* accuracy on the shallower benchmark while trending toward harming it on the deeper one. The asymmetry, not the pooled average, is the result.

1.6.4 The Echo Attractor as a Named Failure Mode

We identify and characterise a recurring failure in which multiple independent systems converge on the same wrong answer — typically the question's own topic entity, an intermediate node from the reasoning chain, or a coarser-granularity container. Because it is shared across systems, it is invisible to any evaluation that treats systems as independent.

1.6.5 Quantified Benchmark Defect Rates for WebQSP and CWQ

We quantify label defects in the two standard benchmarks through a consensus pre-pass followed by per-item adjudication, and separate two kinds of label error that have opposite evidence signatures. We report results both with and without the affected items.

1.6.6 An Evaluation Protocol with Pre-Registered Thresholds

Decision thresholds — the judge-agreement bar, the baseline-certification diagnostic, and the scoring hyperparameters — were fixed before test data was touched, and we report outcomes against them including where a threshold was narrowly missed.

1.7 Scope and Delimitations

The knowledge environment is a static, curated subset of Freebase [18]; the thesis does not address graph construction from unstructured text, knowledge-base completion, or temporal evolution of facts. Questions are English and factoid, drawn from WebQSP [19] and ComplexWebQuestions [20]; long-form generation and conversational settings are out of scope. All systems share one backbone model, held constant so that architectural differences are not confounded with model capability; the thesis therefore reports how these architectures compare on a single backbone, not how they scale across backbones. Finally, the system is a research prototype evaluated offline: latency is reported as a cost measure, not as evidence of production readiness.

1.8 Thesis Organization

Chapter 2 establishes the background required for the technical chapters: knowledge graphs and their Freebase-specific idiosyncrasies, knowledge graph question answering and its evaluation conventions, language models as reasoning engines, retrieval augmentation, graph search, and the statistical tools used throughout. Chapter 3 surveys prior work and positions AGR against it, culminating in a comparison of agentic knowledge graph question answering systems that both identifies the gap this thesis addresses and justifies the choice of baselines.

Chapter 4 describes the construction and validation of the knowledge environment, including the answer-reachability gate that bounds the accuracy attainable in it. Chapter 5 specifies the AGR framework: the tool API, the state machine, the planner, the scoring function, the backtracking policy, and the budgets that guarantee termination. Chapter 6 treats the Structural Verification Layer in detail, as the thesis's principal architectural contribution.

Chapter 7 sets out the experimental design — test sets, baselines, metrics, judge validation, gold-answer quality control, and the pre-registration policy. Chapter 8 reports results and answers the three research questions. Chapter 9 analyses the failures that remain, including the benchmark defects and the shared-attractor pattern, and discusses threats to validity. Chapter 10 summarises the contributions, states the limitations, and identifies future work.

Chapter 2

Background and Preliminaries

This chapter establishes the concepts and notation the technical chapters assume. It is deliberately compact: only material actually used later is included, and standard results are stated rather than derived.

2.1 Knowledge Graphs

2.1.1 Triples, Entities, and Relations

A *knowledge graph* $G = (E, R, T)$ consists of a set of entities E, a set of relation types R, and a set of triples $T \subseteq E \times R \times E$. A triple (h, r, t) asserts that relation r holds from head entity h to tail entity t — for example, (Ada Lovelace, place_of_birth, London). Relations are directed and typed, and both directions carry meaning: the inverse of place_of_birth is a legitimate but distinct relation.

Two properties matter for what follows. Triples are *atomic*: each asserts exactly one fact, so a claim can be checked against the graph by testing for the existence of specific edges. And they are *composable*: a chain $(e_0, r_1, e_1), (e_1, r_2, e_2), \ldots$ expresses a multi-step relationship that no single triple states, which is what makes a knowledge graph the natural substrate for multi-hop questions.

2.1.2 Freebase: MIDs, Schema, and Mediator Nodes

Freebase [18] is a large collaboratively-built knowledge base, frozen since 2016, and the substrate for the standard knowledge graph question answering benchmarks. Three of its characteristics affect any system built on it.

Entities are identified by opaque *machine identifiers* (MIDs) such as m.02mjmr, with human-readable names attached as a separate property. Names are neither unique nor stable, so entity

identity and entity surface form must be handled separately.

Relations are namespaced paths such as people.person.place_of_birth, encoding domain, type, and property. The vocabulary is large — tens of thousands of relation types — and includes many that carry no factual content, such as internal type and key predicates.

Most consequentially, Freebase represents n-ary facts through *mediator* or *compound value type* (CVT) nodes. A fact with more than two participants — an actor playing a particular character in a particular film — is not a single edge but an anonymous intermediate node linking the participants. A relationship a user would describe as one hop is therefore two edges in the graph. Any system reporting hop counts over Freebase must state how it treats these nodes, since the choice changes every depth measurement; systems that ignore them lose the facts they encode [11].

2.1.3 Property Graphs, Neo4j, and Cypher

The triple model above is the RDF view. An alternative is the *property graph* model, in which nodes and edges are both typed and may carry arbitrary key–value properties. This is the model implemented by Neo4j [21] and queried by Cypher [22], a declarative language whose MATCH clause expresses graph patterns directly. A triple store maps onto it naturally: entities become nodes carrying their MID and name, and relations become typed edges. Traversal patterns that would require recursive joins in a relational schema are expressed as literal path patterns, which is the practical reason for choosing a graph store here.

2.2 Knowledge Graph Question Answering

2.2.1 Multi-Hop Questions and Constraint Satisfaction

Knowledge graph question answering (KGQA) takes a natural-language question and a graph, and returns the entity or entities that answer it. A question is k*-hop* if answering it requires traversing k edges from the entities named in the question — its *topic entities* — to the answer.

Beyond hop count, questions vary in *constraints*: conditions that filter candidates rather than extending the chain. “Which of Tolstoy’s novels was published after 1875?” is one hop plus a temporal constraint. Constraints must be satisfied jointly with the chain, and a system that traverses correctly while silently dropping a constraint returns a superset of the answer — a failure that hop-count analysis alone does not expose.

Two families of approach have historically divided this field. *Semantic parsing* methods translate the question into a formal query (SPARQL or equivalent) and execute it, giving exact and interpretable results but requiring the parse to be exactly right [19, 23]. *Information retrieval* methods retrieve a relevant subgraph and rank candidate answers within it, degrading more

gracefully but sacrificing transparency. The agentic systems considered in this thesis are closer to the second family, but recover much of the first family's transparency by logging the traversal itself.

2.2.2 Evaluation Conventions: Hits@1 and F1

Two metrics are conventional. *Hits@1* is the proportion of questions for which the system's top-ranked answer appears in the gold answer set; it is a per-question binary correctness measure and the headline number in most of this literature.

Because many questions admit multiple correct answers, Hits@1 alone rewards a system that names one correct entity and ignores the rest. *F1* therefore complements it, computed per question between the predicted answer set A and the gold set A^*:

$$P = \frac{|A \cap A^*|}{|A|}, \qquad R = \frac{|A \cap A^*|}{|A^*|}, \qquad F_1 = \frac{2PR}{P+R},$$

and averaged over questions. The distinction between P and R is load-bearing in this thesis: a system that asserts many entities can achieve high recall while its precision falls, and Section 1.3 argues that precisely this failure goes unmeasured when only Hits@1 is reported.

2.3 Large Language Models as Reasoning Engines

2.3.1 In-Context Learning and Chain-of-Thought Prompting

Sufficiently large language models perform tasks specified entirely in their prompt, without gradient updates — *in-context learning* [24]. Prompting the model to produce intermediate reasoning steps before its final answer, *chain-of-thought* prompting [25], substantially improves performance on multi-step problems. Both are used here: the agent's planner, scorer, evaluator, and verifier are all prompted components of a single frozen model, and no component is fine-tuned. This matters for comparability, since several systems surveyed in Chapter 3 are fine-tuned and therefore not directly comparable to prompting methods.

2.3.2 Tool Use and the ReAct Loop

A language model can be given *tools*: named operations it may invoke, whose results re-enter its context [26]. The *ReAct* pattern [27] interleaves reasoning and action in a loop — the model reasons about what to do, acts, observes the result, and reasons again — which is the control structure underlying agentic graph navigation. Where the tools are graph operations, each

iteration extends a traversal, and the sequence of tool calls is a complete record of how the answer was reached.

2.3.3 A Working Definition of Hallucination for This Thesis

"Hallucination" is used loosely in the literature. This thesis adopts a narrow, operational definition tied to the knowledge environment: an asserted entity is *ungrounded* if it does not exist in the graph, or exists but is not reachable from the question's topic entities within the traversal budget. A claim is *unsupported* if the traversed triples do not entail it.

Two consequences of this definition should be stated plainly. It is relative to the environment: an entity absent from the graph is ungrounded even if true in the world. And groundedness is not correctness — an answer can be perfectly grounded, every entity real and reachable, and still be the wrong answer to the question. Chapter 8 shows that this gap is not hypothetical, and Section 1.3 is built on it.

2.4 Retrieval-Augmented Generation

2.4.1 Dense Retrieval and Sentence Embeddings

Dense retrieval encodes queries and documents into a shared vector space and retrieves by nearest neighbour, capturing semantic similarity where lexical matching fails [28]. Sentence-level encoders such as Sentence-BERT [29] produce embeddings suitable for direct cosine comparison. In this thesis, embeddings serve two bounded purposes — linking question phrases to graph entities, and pre-ranking relation candidates during exploration. They never establish that a fact is true; that role belongs exclusively to the symbolic triples.

2.4.2 Graph-Based Retrieval

Graph-based retrieval replaces flat passages with structured context [4, 5]. The retrieved unit is a subgraph rather than a document, which preserves relationships explicitly. The distinction that matters for this thesis is not graph versus vector but *static versus interleaved*: whether retrieval happens once before reasoning begins, or repeatedly as reasoning proceeds.

2.5 Search over Graphs

2.5.1 Best-First and Beam Search

Best-first search maintains a frontier of candidate nodes ordered by a heuristic estimate of promise and expands the most promising first [30, 31]. *Beam search* bounds the frontier to the b highest-scoring candidates at each depth, trading completeness for a fixed memory and computation budget. With an expensive scoring function — here, a language model call — the beam width is also the principal cost control.

2.5.2 Backtracking and Dead-End Detection

Bounded search commits to choices that may prove wrong. *Backtracking* recovers by abandoning the current branch and resuming from a previously unexplored alternative, which requires maintaining a stack of such alternatives and a trigger condition for when to pop it. Trigger conditions and the stack discipline used here are specified in Section 5.6.

2.5.3 Relation to Monte Carlo Tree Search: A Terminological Clarification

The original proposal for this work described its navigation strategy as "inspired by Monte Carlo Tree Search". That description is withdrawn here, and the reason is worth stating explicitly.

Monte Carlo Tree Search [32, 33] is defined by four phases: selection, expansion, *simulation* (a rollout to a terminal state), and *backpropagation* of the rollout's value to the ancestors of the expanded node. The two phases that give MCTS its statistical character are simulation and backpropagation, which together let node values be estimated from sampled outcomes rather than from a fixed heuristic.

The method used in this thesis has neither. There are no rollouts, and no value is propagated back up the search tree; each candidate is scored once, by a fixed function combining embedding similarity with a language-model judgement, and that score is never revised in light of what is found later. What the method is, is *guided best-first search with a bounded beam and backtracking*. It is described that way throughout. Related work has applied genuine tree-search methods to knowledge graph reasoning [34]; conflating that with the present approach would overstate what is implemented here.

2.6 Statistical Tools Used in This Thesis

Three tools recur, each chosen for a specific property of the data.

Bootstrap confidence intervals [35] quantify sampling uncertainty by resampling the evaluated questions with replacement and recomputing the metric; the 2.5th and 97.5th percentiles of the resulting distribution give a 95% interval. They are used because the test sets are samples, and because they require no distributional assumption about metrics such as per-question F1.

McNemar's test [36] assesses whether two systems differ in accuracy when both are evaluated on the *same* questions. It considers only the discordant pairs — questions one system answers correctly and the other does not — and tests whether the split between the two discordant counts departs from chance. Paired testing is essential here because all systems are run on identical question samples, so an unpaired test would discard the pairing and lose power.

Cohen's κ [37] measures agreement between two annotators on a categorical judgement, correcting for agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is observed agreement and p_e is the agreement expected if both annotators labelled independently at their observed marginal rates. It is used to validate the automatic groundedness judge against human annotation. Conventional interpretation follows Landis and Koch [38], on whose scale values between 0.61 and 0.80 denote substantial agreement.

Chapter 3

Related Work

This chapter surveys the work AGR builds on and competes with. It proceeds from static graph-augmented generation, through path-retrieval and agentic exploration, to verification and factuality measurement, and closes with a structured comparison of the six closest systems. That comparison serves two purposes: it locates the gap this thesis addresses, and it justifies the choice of baselines in Section 7.4.

3.1 Static Graph-Augmented Generation

3.1.1 GraphRAG and Query-Focused Summarization

GraphRAG [4] constructs a knowledge graph from a text corpus using a language model, partitions it into communities, and pre-computes summaries at each level, so a query can be answered from structured context rather than retrieved passages. Its strength is global questions — those whose answer depends on the corpus as a whole rather than any single passage — which flat retrieval handles poorly.

The relevant limitation is that retrieval remains a single step preceding generation. The context is assembled before the model has begun reasoning, so it cannot be conditioned on intermediate findings. GraphRAG also builds its graph by language-model extraction, which is appropriate for unstructured corpora but introduces extraction error into the reference itself — a consideration that shapes the environment design in Section 4.1.

3.1.2 HippoRAG and Memory-Structured Retrieval

HippoRAG [5] draws on the hippocampal indexing theory of human memory, combining a knowledge graph with a personalised PageRank retrieval step so that a single query can integrate evidence distributed across many passages. It substantially improves multi-hop retrieval over flat baselines at low cost.

Its retrieval is nonetheless a single graph-propagation step, not an agent-directed traversal: the propagation is fixed and the model does not choose where to look next. Temporal extensions such as T-GRAG [6] address a different axis — conflicting and redundant facts across time — which is orthogonal to the navigation question and is noted here for completeness rather than compared against.

3.2 Path-Retrieval and Semantic-Parsing KGQA

3.2.1 Reasoning-on-Graphs

Reasoning-on-Graphs (RoG) [9] adopts a planning–retrieval–reasoning pipeline: a fine-tuned model generates relation paths grounded in the graph schema, those paths are used to retrieve matching instantiations, and the model reasons over the retrieved paths. Because answers are produced from retrieved paths, RoG is faithful by construction and can present the path as an explanation.

Two properties distinguish it from the present work. It is *fine-tuned*, which places its reported numbers outside direct comparison with prompting methods. And its plan is generated in one shot before retrieval: if the generated relation path does not instantiate in the graph, there is no mechanism to revise it mid-course. The pre-generation verification question does not arise, because grounding is guaranteed structurally rather than checked.

3.2.2 StructGPT and KG-Agent

StructGPT [12] provides a general interface for reasoning over structured data — knowledge graphs, tables, and databases — through an invoke–linearise–generate loop, in which specialised interfaces extract evidence that is linearised into text for the model. It is deliberately general rather than graph-specific, and performs no open-ended graph search.

KG-Agent [10] is closer to the agentic setting: a multifunctional toolbox, a knowledge-graph executor, and a dynamic memory, driven by an instruction-tuned model that autonomously selects tools across iterations. It demonstrates that a small fine-tuned model with good tool abstractions is competitive with much larger prompted ones. It plans implicitly through tool selection rather than committing to explicit sub-objectives, provides no backtracking mechanism, and performs no verification of the final answer.

3.3 Agentic Graph Exploration

3.3.1 Think-on-Graph

Think-on-Graph (ToG) [7] established the template for training-free agentic KGQA. The model performs iterative beam search over the graph: at each step it is shown candidate relations and entities and prunes them itself, with beam width and maximum depth both set to three. Exploration terminates when the model judges the retrieved paths sufficient, at which point it answers directly.

ToG is the most directly comparable prior system to AGR — training-free, prompting-based, operating on Freebase, evaluated on the same benchmarks — and is therefore the agentic baseline in Section 7.4. Its limitations are instructive. There is no explicit decomposition, so multi-constraint questions are handled implicitly. Low-scoring branches are pruned within the beam, but there is no mechanism to return to an abandoned node once the beam has moved on. And the transition from exploration to answering is a single model judgement: nothing checks the answer that follows.

3.3.2 Plan-on-Graph

Plan-on-Graph (PoG) [8] is the closest system to AGR in ambition. It decomposes the question into sub-objectives including constraint conditions, explores adaptively with a breadth that varies by sub-objective, and adds Guidance, Memory, and Reflection mechanisms. Reflection decides both whether to self-correct and which entity to backtrack to, making PoG the one surveyed system with genuine backtracking.

PoG's self-correction operates on the *search process*: it reconsiders where to explore. It does not decompose the draft answer into claims and check each against the retrieved triples before responding. The distinction is the gap this thesis occupies, and it is stated precisely in Section 3.7.

A naming hazard must be recorded here. A second, unrelated system is also abbreviated PoG — Paths-over-Graph [39] — and reports considerably higher figures. Survey tables that cite "PoG" without disambiguation are a known source of contaminated comparisons, and this thesis always writes Plan-on-Graph or Paths-over-Graph in full.

3.3.3 Generate-on-Graph and Reasoning with Trees

Generate-on-Graph (GoG) [11] targets the incomplete-graph setting through a thinking–searching–generating loop: when the graph lacks a required fact, the model generates candidate triples from its parametric knowledge and continues. It also expands subgraphs dynamically to handle the mediator nodes described in Section 2.1, which ToG tends to skip. The trade-off

is directly opposed to this thesis's: GoG deliberately admits model-generated facts into the evidence, whereas AGR admits only graph triples.

Reasoning with Trees [34] applies explicit tree search to knowledge graph question answering. It is cited in Section 2.5.3 as an example of genuine tree-search machinery, in contrast with the guided best-first search implemented here.

3.4 Verification and Self-Correction in Language Models

3.4.1 Chain-of-Verification

Chain-of-Verification (CoVe) [15] has the model draft a response, plan verification questions about it, answer those independently, and regenerate a corrected response. It establishes the core premise of this thesis's verification layer — that decomposing a draft and checking its parts reduces factual error — but performs the checks against the model's own knowledge rather than an external symbolic source. Multi-agent debate [16] achieves a related effect by having several model instances critique one another.

This distinction matters given evidence that language models cannot reliably self-correct reasoning without external feedback [40]. A verification step whose ground truth is the same model that produced the error is limited by construction. AGR's verification is grounded in traversed triples, making the check external to the generator.

3.4.2 Ontology-Guided and Self-Correcting Graph RAG

A self-correcting agentic graph RAG system for clinical decision support in hepatology [17] demonstrates the combination of agentic retrieval with a correction loop in a high-stakes domain, and ontology-guided reasoning constrains an agent to logically valid relation use. These substantiate the practical motivation for verification, though in domain-specific settings rather than on the open-domain benchmarks used here. Reflexion-style verbal self-improvement [41] provides the general agentic pattern of which these are graph-specific instances.

3.5 Measuring Factuality

3.5.1 Claim-Decomposition Metrics

FActScore [42] evaluates long-form generation by decomposing text into atomic facts and verifying each against a reference source, reporting the supported proportion. This decomposition-then-verify protocol is the basis for the groundedness metric in Section 7.5.3, with the knowledge

graph serving as the reference. The essential methodological requirement, adopted here, is that the verifying judge must be independent of the system that produced the text.

3.5.2 Factuality Benchmarks and Their Limits

FactBench [43] provides a dynamic benchmark for in-the-wild factuality evaluation. The original proposal for this thesis intended to use it both as a source for the knowledge environment and as the hallucination evaluator. That plan was abandoned, and the reason is methodological rather than practical: if the evaluator's facts are contained in the graph the system retrieves from, a low hallucination rate follows by construction rather than by merit. The evaluation in Chapter 7 therefore measures claim grounding against the graph with a human-validated judge, keeping the evaluator's ground truth outside the system's evidence.

3.6 Comparative Summary of Agentic KGQA Systems

Table 3.1 compares the six closest systems on the mechanisms relevant to this thesis, and Table 3.2 records their evaluation settings and published accuracy.

Three caveats attach to Table 3.2 and are carried into every comparison in this thesis.

First, the figures are drawn from the original publications and reflect different backbones. RoG and KG-Agent are fine-tuned, and their numbers are not comparable with prompting methods at all. Among the prompting methods, the backbone accounts for much of the spread — the same methods drop sharply when run on smaller open-weight models. This is the direct motivation for holding the backbone constant across every system in Chapter 7: the literature does not, so a cross-paper table cannot settle which architecture is better.

Second, evaluation protocols differ. Some papers evaluate on sampled test subsets, and answer-matching conventions are not uniform across implementations.

Third, the Plan-on-Graph figures are marked as approximate. They are consistent with values reported in follow-up work, but should be confirmed against the primary tables before being relied upon in any quantitative claim. No claim in this thesis depends on them; they appear here to situate the design space.

3.7 Positioning of This Work

Table 3.1 makes the gap explicit. Decomposition is well established — PoG and GoG both do it. Backtracking exists, in PoG. Faithful grounding by construction exists, in RoG. But in the final column, *no surveyed system decomposes its draft answer into atomic claims and checks each against the retrieved triples before responding*. PoG's Reflection corrects the search while it is

Table 3.1: Mechanism comparison of the six closest prior systems. No system performs claim-level verification of the final answer against retrieved triples before responding.

System	Exploration strategy	Decomposes query?	Backtracks?	Verifies before answering?
ToG [7]	Training-free LLM-guided iterative beam search; beam width and depth both 3	No	No — prunes within beam, cannot return	No
PoG [8]	Self-correcting adaptive planning with Guidance, Memory, Reflection	Yes, into sub-objectives with constraints	Yes — Reflection selects the entity to return to	Partial — corrects the search, not the answer
RoG [9]	Planning–retrieval–reasoning over generated relation paths	Partial — relation-path plans	No	Faithful by construction; no explicit check
KG-Agent [10]	Toolbox with KG executor and dynamic memory; autonomous tool selection	Implicit, via tool calls	No	No
GoG [11]	Thinking–searching–generating; generates triples when the KG is incomplete	Yes	No — expands subgraph instead	No
StructGPT [12]	Iterative reading-then-reasoning via invoke–linearise–generate interfaces	No	No	No
AGR (this work)	Guided best-first beam search with explicit backtracking	Yes, into ordered sub-objectives	Yes — threshold, dead end, or evaluator veto	**Yes — claim-level, pre-generation**

Table 3.2: Reported evaluation settings and Hits@1 as published. These figures are *not* directly comparable to one another: backbones differ, some papers evaluate on sampled subsets, and the backbone accounts for much of the variation.

System	Backbone	Datasets	WebQSP	CWQ
ToG	GPT-3.5	CWQ, WebQSP, GrailQA [44], QALD10-en, SimpleQuestions, WebQuestions, T-REx, Zero-Shot RE, Creak	76.2	58.9
	GPT-4		82.6	69.5
PoG	GPT-3.5	CWQ, WebQSP, GrailQA	≈82.0	≈63.2
	GPT-4		≈87.3	≈75.0
RoG	Fine-tuned LLaMA2-7B	WebQSP, CWQ	85.7	62.6
KG-Agent	Fine-tuned 7B	WebQSP, CWQ, plus other KBs	83.3	72.2
GoG	GPT-3.5	WebQSP, CWQ (complete and incomplete KG)	78.7	55.7
	GPT-4		84.4	75.2
StructGPT	GPT-3.5	WebQSP, MetaQA, TableQA, Text-to-SQL	72.6	—

running; RoG's answers are grounded because of how they are produced, not because anything examined them. The slot is open.

AGR's contribution is therefore positioned as follows.

The **primary** contribution is the Structural Verification Layer: a claim-level check against traversed triples, executed before the answer is emitted, with repair policies that either resume exploration or remove unsupported content. Chapter 6 specifies it.

The **secondary** contribution is a systematic component ablation quantifying what each agentic mechanism actually contributes, in accuracy and in tokens. As Table 3.1 shows, these mechanisms are typically reported as bundles; which of them earns its cost is not established in this literature.

A deliberate consequence of this positioning is that the thesis does not depend on beating prior systems on raw Hits@1. The comparison that matters is against the same architecture with the verification layer removed, on the same graph, with the same backbone — which is why the ablation in Section 8.7, rather than Table 3.2, carries the argument. Where AGR is compared against a prior system, that comparison is run under controlled conditions in this thesis's own environment rather than quoted across papers.

Chapter 4

The Knowledge Environment

Everything the agent can know, it knows from the graph. The environment therefore bounds the accuracy any system in this thesis can attain, and a result reported without that bound is uninterpretable. This chapter describes how the environment was built, what it contains, and — equally important — what it cannot express.

4.1 Design Goals

4.1.1 Decoupling the Graph Source from the Question Sets

The original proposal for this work described ingesting WebQSP and FactBench into a graph database. That description conflated two distinct roles, and the correction is worth stating because it shaped everything downstream.

WebQSP [19] is not a knowledge graph. It is a set of questions annotated with semantic parses over Freebase [18]; the graph is Freebase, and WebQSP supplies questions and gold answers against it. The two roles are kept strictly separate here: Freebase-derived triples constitute the environment, while WebQSP and ComplexWebQuestions [20] supply questions and gold answers and contribute no facts to it.

FactBench [43] was removed entirely. It had been intended both as a source of graph content and as the hallucination evaluator, which is circular: if the evaluator's facts are inside the graph the system retrieves from, a low hallucination rate follows by construction. Groundedness is instead measured against the graph by an independent judge, as described in Section 7.6.

4.1.2 Why a Curated Subgraph Rather Than LLM-Extracted Triples

The proposal also specified a language-model entity-relationship extractor to build the graph. This is appropriate when the source is unstructured text, and it is what GraphRAG does [4]. It is

self-defeating here. The graph is the reference against which model output is verified; building that reference with a language model would introduce the very error class the thesis measures, and no claim about hallucination reduction would survive the observation that the ground truth was itself generated.

The environment is therefore assembled exclusively from curated Freebase triples. Language models are used to *navigate* the graph and to *judge* answers, never to populate it.

4.2 Source Data

4.2.1 The Freebase Snapshot

Freebase has been frozen since 2016, which makes it stable and reproducible while also dating it — a property that cuts both ways and motivates the no-retrieval control in Section 7.4. Two routes to a working Freebase environment were considered.

The first is a full SPARQL endpoint: the community-standard preprocessed Virtuoso image, which ships a database of over 53 GB and whose maintainers recommend on the order of 100 GB of RAM for acceptable query latency. Its advantage is a complete and realistic search space, including the hub entities of very high degree that make naive traversal explode.

The second is the per-question subgraph distribution published alongside Reasoning-on-Graphs [9], in which each record pairs a question with its topic entities, gold answers, and the triples of a multi-hop neighbourhood around those topic entities.

The second route was chosen, for a reason that should be recorded plainly: the hardware for the first was not available. The consequence is a scoped environment rather than all of Freebase, and Section 4.7 quantifies what that costs.

4.2.2 WebQSP and ComplexWebQuestions as Question Sets

WebQSP [19] contains questions whose answers lie predominantly one or two hops from the topic entity; it descends from WebQuestions [23]. ComplexWebQuestions [20] is constructed by composing WebQSP queries into deeper chains with conjunctions, superlatives, and comparatives, and is the genuine multi-hop benchmark of the two. Reporting both is deliberate: WebQSP establishes comparability with prior work, and CWQ is where the multi-hop claim is actually tested.

4.3 Graph Construction

4.3.1 Union of Per-Question Subgraphs from the RoG Distribution

The environment is the *union* of every per-question subgraph in the RoG distribution, across both datasets and all three splits. Each record's triple list is read, whitespace-normalised, and inserted into a global deduplicated set; entity and relation strings are interned to integers so that the union and its adjacency structure fit in memory.

Unioning is not incidental. A per-question subgraph in isolation is a near-oracle: it was extracted to contain the answer, so an agent searching it faces almost no distractors. Merging every question's neighbourhood into one global graph restores distractor mass, since the agent exploring one question's topic entity encounters edges contributed by unrelated questions. WebQSP contributes 3,791,302 unique triples over 1,298,304 entities; adding CWQ brings the union to 8,309,194 triples over 2,592,892 entities.

Two consequences must be disclosed. First, the RoG subgraphs were pre-extracted so as to guarantee answer reachability, which makes the resulting environment friendlier than raw Freebase — fewer irrelevant hub explosions, and a higher answer-reachability rate than a BFS extraction from the full store would produce. This is a threat to external validity, revisited in Section 9.9.3. The mitigating argument is that *every system compared in this thesis navigates this identical environment*, so relative comparisons remain sound even where absolute numbers are optimistic.

Second, the distribution's triples are keyed by human-readable entity names rather than by Freebase machine identifiers. The node identifier is therefore the name string, which means distinct real-world entities sharing a surface form are merged into a single node. This is inherent to name-keying and is also revisited in Section 9.9.3.

4.3.2 Relation Filtering and Noise Removal

An extraction from raw Freebase would require aggressive predicate filtering: administrative namespaces, type and key predicates, and above all type.type.instance, which contributes millions of edges per type node. Degree capping would likewise be mandatory, since uncapped breadth-first expansion from a hub entity explodes at the second hop.

No such filtering was applied here, and the reason is structural rather than an oversight: the RoG distribution ships subgraphs that have already been scoped around topic entities, so the administrative predicates and hub explosions that filtering exists to remove are largely absent from the source. The only transformation applied to relation names is sanitisation into Cypher-legal identifiers, described in Section 4.4. What survives is a vocabulary of 7,058 distinct relation types, which is the relation search space the agent faces at every exploration step.

4.3.3 Treatment of Mediator Nodes and Its Effect on Hop Counts

Freebase reifies n-ary facts through mediator nodes, as described in Section 2.1. In the name-keyed distribution these surface distinctively: named entities appear as their names, while unnamed mediators survive as raw machine identifiers of the form m.0d05w3 or g.11b62t84jq. Nodes are flagged as mediators by matching that pattern, and the flag is stored as an is_cvt property.

Mediators are *retained*, not collapsed into hyper-edges. Retaining them preserves the facts they encode — systems that skip mediators lose exactly those facts [11] — at the cost of inflating raw hop counts, since one logical hop through a mediator is two graph edges. Every hop budget in this thesis is therefore stated in *raw* hops, and the conversion is roughly two raw hops per logical hop.

The scale of this decision is easy to understand: **1,652,618 of the 2,592,892 nodes — 63.7% — are mediator-form nodes carrying no human-readable name.** Nearly two thirds of the environment is connective tissue rather than nameable entities, which is why the tool layer in Section 5.2 must traverse through mediators transparently rather than present them to the agent as candidate answers.

4.4 Graph Store Construction

4.4.1 Property-Graph Schema

Entities become :Entity nodes carrying an integer id, the name string, and the boolean is_cvt flag. Triples become typed relationships whose type is the sanitised relation name — non-alphanumeric characters replaced by underscores, since Freebase predicates contain dots that are illegal in unquoted Cypher relationship types — with the original dotted string retained as an fb_name property.

Native relationship types were chosen over a single generic type carrying a name property. The alternative simplifies import but slows filtered expansion, and filtered expansion is the agent's hottest operation: every exploration step retrieves the relations of an entity and then the neighbours along one of them.

Integer identifiers were chosen over names as the import key. Entity names contain commas, quotation marks, and unicode, all of which invite escaping bugs and silent collisions in bulk-import CSVs. The name survives as an ordinary property and is what the entity resolver matches against.

Table 4.1: The knowledge environment as constructed and imported.

Property	Value
Entities (nodes)	2,592,892
Triples (relationships)	8,309,194
Node and relationship properties	16,087,870
Distinct relation types	7,058
Mediator-form nodes (is_cvt)	1,652,618 (63.7%)
Triples contributed by WebQSP subgraphs	3,791,302
Entities after WebQSP alone	1,298,304
Rows skipped during import	0
Import wall-clock time	36.4 s
Peak import memory	1.09 GiB
Neo4j version	Community 5.26.28

4.4.2 Bulk Import into Neo4j

Import used the offline bulk importer of Neo4j Community 5.26.28, which operates on a stopped database and is orders of magnitude faster than transactional loading at this scale. Duplicate nodes and bad relationships were configured to be skipped rather than to abort the import.

After import, a uniqueness constraint on Entity.id and a full-text index on Entity.name were created; the latter is what the resolver's lexical tier queries.

4.4.3 Graph Statistics

Table 4.1 records the environment as built. These figures, together with the dataset versions and the hop cap, constitute the reproducibility record for the environment.

The zero skipped rows deserve emphasis, because the import was deliberately configured to tolerate skips. Node and relationship counts in the database match the CSV row counts exactly, so the count-reconciliation check of Section 4.7 passes without any unexplained gap to account for.

4.5 The Vector Index

4.5.1 Entity Embeddings

Entity names were embedded with all-MiniLM-L6-v2, a 384-dimensional sentence encoder [29] chosen for CPU throughput at this node count. Mediator nodes were skipped: embedding a string such as m.0y5k79m produces noise, and by Section 4.3 that is almost two thirds of the graph, so skipping them is also the difference between a tractable and an intractable embedding job. Vectors were written back into Neo4j and indexed with a cosine-similarity vector index.

4.5.2 Relation-Vocabulary Embeddings

The scoring function of Section 5.5 compares candidate relations against the current sub-objective in embedding space, so the 7,058 relation names are embedded once and held in memory rather than embedded per step.

Two details make this work. Relation names are *verbalised* before encoding: people.person.place_of_birth becomes "people person place of birth", with consecutive duplicate tokens removed. The encoder was trained on natural language, and the verbalised phrase lands near a question like "where was X born" in embedding space where the raw dotted path does not. And relations are embedded with the *same* model as entities and sub-objectives, since vectors from different models occupy unrelated spaces and their cosine similarity is meaningless.

The resulting artefact is a 7058×384 float32 matrix with a parallel name list. Because vectors are L2-normalised at encoding time, similarity against the whole vocabulary is a single matrix–vector product.

A functional check confirms the verbalisation is doing its job. Encoding the probe "where was the person born" and ranking the vocabulary against it returns people.person.place_of_birth first at cosine 0.709 and location.location.people_born_here — the inverse relation — second at 0.675, with the remaining top-five entries plausible near-misses drawn from birth-related and fictional-character namespaces. A degenerate verbaliser would have produced an unordered top five here.

4.5.3 Scope: Linking and Candidate Ranking, Never Ground Truth

The role of embeddings in this thesis is deliberately narrow. They resolve question phrases to graph entities, and they pre-rank relation candidates so the language-model scorer can be applied to a short list rather than to thousands of relations. They never establish that a fact holds. Every factual check — whether a triple exists, whether an entity is reachable, whether a claim is supported — is answered by symbolic graph queries. Nothing in the verification layer of Chapter 6 consults an embedding.

4.6 Entity Resolution and Surface-Form Linking

4.6.1 The Exact, Lexical, and Vector Cascade

Mapping a question's surface mention to a graph node uses a three-stage cascade, each stage attempted only if the previous one fails.

Exact matching queries for a node whose name equals the normalised mention. Because the environment is name-keyed and the benchmarks supply topic entities as names, this resolves

Table 4.2: Entity resolution over all topic-entity mentions in both test sets. A mention counts as resolved only when the returned node's name equals the mention exactly.

Dataset	Mentions	Exact stage	Unresolved
WebQSP	1,661	1,661 (100%)	0
CWQ	5,488	5,478 (99.8%)	10
Total	**7,149**	**7,139 (99.86%)**	**10 (0.14%)**

nearly everything.

Lexical matching queries the full-text index, handling typographical variation and partial names. Mentions are escaped for Lucene syntax before the query: benchmark entity names contain parentheses, colons, and slashes, all of which are Lucene operators that would otherwise crash or silently distort the query.

Vector matching encodes the mention and queries the entity vector index, handling paraphrases and misspellings that survive the first two stages.

The cascade is referred to by these names throughout, never as "tiers", which this thesis reserves for the two-tier groundedness metric of Section 7.5.3.

4.6.2 Threshold Selection on Development Data

The lexical stage accepts its top hit only when the full-text score exceeds 1.0, falling through to vector matching otherwise. The threshold exists because the full-text index returns a ranked list for almost any input, including inputs that match nothing meaningfully; without a floor, the lexical stage would confidently return the least-bad token overlap and the vector stage would never be reached.

Resolution was measured over every topic-entity mention in both test sets, and Table 4.2 reports the outcome.

Entity linking is therefore not a bottleneck in this environment, and it can be excluded as an explanation for downstream failures — a claim the error analysis of Chapter 9 relies on.

4.7 Environment Validation Gate

No experiment was run until the environment was certified. The gate has three parts of increasing strength.

4.7.1 Answer-Reachability Protocol

For every test question, two properties are computed over the union graph. *Existence* asks whether the gold answer entities appear as nodes at all. *Reachability* asks whether at least one

Table 4.3: Environment coverage over the full test sets, at a bound of four raw hops. The reachability column is the Hits@1 ceiling for every system evaluated in this thesis.

Dataset	Test questions	All answers exist	≥1 exists	≥1 reachable
WebQSP	1,628	94.7%	97.3%	**97.3%**
CWQ	3,531	99.4%	99.7%	**99.7%**

gold answer is connected to at least one topic entity within a bounded number of hops.

Reachability is computed by breadth-first search treating edges as *undirected*, because the agent's neighbour-retrieval tool looks in both directions and the gate must model the agent's actual traversal semantics. The bound is four raw hops, which by Section 4.3 is approximately two logical hops through mediators — enough for all of WebQSP and most of CWQ. A question whose topic entity is itself a gold answer counts as reachable at zero hops.

This number is the **accuracy ceiling**: if a gold answer is unreachable, no navigating system can find it, however well designed.

4.7.2 Coverage Results and the Induced Accuracy Ceiling

Table 4.3 reports the gate.

Three observations follow. Every question in both datasets has at least one topic entity present in the graph, so no question is lost at the entry point.

More strikingly, *existence and reachability are identical* — 1,584 of 1,628 for WebQSP and 3,519 of 3,531 for CWQ. Every gold answer that exists in this environment is reachable within four hops. Reachability is therefore never the binding constraint; when an answer is unavailable it is because the entity is absent altogether. This is a direct consequence of the construction in Section 4.3: the subgraphs were extracted around topic entities precisely to guarantee reachability, and the measurement confirms that the property survived unioning.

Finally, CWQ's coverage exceeds WebQSP's, which inverts the expected ordering — the deeper benchmark has the better-covered answers. The explanation is again constructional rather than semantic: CWQ contributes far more questions and therefore far more subgraphs to the union, so its own answers are better represented. The residual WebQSP shortfall is dominated by answers that are not entities at all, which Section 4.7.4 takes up.

4.7.3 Analysis of Linking Misses and Gate Failures

The ten unresolved mentions of Table 4.2 are not a random residue. Every one is a bare machine identifier of the form `g.123852dg` appearing as a CWQ topic entity — an unnamed mediator node that the benchmark supplies where a named entity was expected. Exact matching fails because no node carries that string as a *name*, and the lexical stage then returns a meaningless

single-character match. The correct characterisation is that these questions are anchored on nodes with no surface form, not that resolution is unreliable.

The strongest gate check re-verifies the Python-computed reachability against the live database, confirming that nothing was lost between the CSV files and the imported graph. A random sample of 400 questions flagged reachable, drawn under a fixed seed, was re-tested with a shortest-path query in Neo4j. The pass criterion is strict — these were already known to be reachable, so anything below 100% would indicate dropped edges. **All 400 verified; the failure set is empty.** Together with the zero skipped import rows and the exact count reconciliation, the environment is certified.

4.7.4 What the Environment Cannot Express

Certifying reachability establishes that answers which *are* entities can be found. It says nothing about answers that are not entities, and this is where the environment's real limits lie.

The import schema of Section 4.4 gives every node a single :Entity label. There is no typed literal node, no datatype, and no distinction between an entity and a value. A date or a number can therefore exist in the graph only incidentally, as an entity whose *name* happens to be a numeric string. Measured over all 2,592,892 nodes:

- **Dates** are effectively absent. Only 77 nodes carry a full YYYY-MM-DD name and 175 a bare four-digit year — together under 0.01% of the graph, and untyped, so they cannot be compared, ordered, or filtered by range.
- **Numeric values** appear on 12,622 nodes (0.49%), but the population is dominated by identifiers such as ISBNs rather than by measurable quantities.
- **Ordinals and superlatives** are not expressible at all. This is a property of the tool layer rather than the data: Section 5.2 exposes no aggregation, sorting, or comparison operator, so "the smallest" or "the earliest" cannot be computed even when every candidate is present in the graph.

This defines a class of questions that is *unanswerable in this environment*: any question whose gold answer is a date, a measured quantity, or a superlative selection over candidates. Such a question is not a reasoning failure when the system misses it — the environment could not have supplied the answer. The class is referenced by name in Section 9.6, where it accounts for a substantial share of the residual failures, and it is a known consequence of a design decision: modelling literals as typed nodes was considered and not adopted, in exchange for a uniform schema and a simpler tool API.

Chapter 5

The AGR Framework: Architecture and Navigation

This chapter specifies the Agentic Graph Reasoning framework: how the agent is structured, what operations it may perform on the graph, how it decides where to look next, how it recovers from wrong turns, and what guarantees bound its cost. The verification layer, which is the thesis's principal contribution, is separated into Chapter 6; everything here is the machinery that produces the traversal it verifies.

5.1 Architectural Overview

5.1.1 The Controlled-Agent Pattern

There are two ways to build a graph-navigating agent. The first hands the model a set of tools through native function-calling and lets it drive the loop itself, ReAct-style [27]. The second inverts control: the *program* orchestrates the loop, calls the graph tools deterministically, and consults the model only at decision points, through constrained prompts that must return JSON. AGR uses the second, and the choice is deliberate enough to be worth defending, because it determines what the word "agentic" means in this thesis. Four consequences follow:

- **Every graph operation is logged by construction.** The traversal is not reconstructed from the model's narration of what it did; it is the literal sequence of tool calls the program made.
- **Budgets are enforced in code, not requested in prompts.** A prompt asking the model to stop after four hops is a suggestion. A counter checked in a router is a guarantee (Section 5.8).

START → **Planner** → **Explorer** → **Evaluator**

Evaluator routes on route_after_eval:

- continue → **Explorer**
- backtrack → **Backtracker** → **Explorer**
- answer → **Verifier**

Verifier routes on route_after_verify:

- grounded → **Answerer**
- retry → **Explorer**
- give_up → **Answerer**

Answerer → END

Figure 5.1: The AGR state machine. Two cycles exist: Explorer ↔ Evaluator for iterative search, and Verifier → Explorer for verification-driven re-exploration. Both are bounded by the budgets of Section 5.8 rather than by the graph structure.

- **Failures are reproducible.** A malformed model response degrades a decision; it cannot corrupt the control flow.
- **Each model call is small, single-purpose, and cacheable**, which is what makes the response cache of Section 5.9 viable.

The agency in AGR therefore lives in the *decision policy* — which sub-objectives to pursue, which edges look promising, whether the current path is a dead end, whether the draft answer is supported — and not in unconstrained tool invocation. Prior systems in this space make the same choice [7, 8]; stating it explicitly forecloses the objection that AGR is merely a prompt around a function-calling loop.

5.1.2 The State Machine

The framework is a directed graph of six nodes over a shared typed state, built with LangGraph. Figure 5.1 gives the topology.

Two properties of this topology matter. It contains *cycles*, which is why a state-machine framework is used rather than a linear pipeline: exploration loops until the evaluator is satisfied, and verification can send the agent back to explore. And the verifier sits *between* the decision to answer and the answer itself, which is the structural expression of this thesis's contribution — there is no path from evaluator to answerer that does not pass through verification.

Table 5.1: The graph tool API. All operations are deterministic parameterised Cypher; none consults an embedding except search_entity, which delegates to the resolver of Section 4.6.

Operation	Returns	Role
search_entity(*s*, k)	up to k candidate entities with id, name, score, and resolver stage	anchor seeding; the only entry point from text to graph
get_relations(*e*)	relation types incident to e in *both* directions, each with its fanout count	candidate generation for the scorer
get_neighbors(*e*, *r*, *d*)	entities reached from e along r in direction d, each with the via chain that reached it	frontier expansion
verify_triple(*h*, *r*, *t*)	{direct, via_cvt, supported}	claim checking (Section 6.3.1)
verify_connection(*a*, *b*)	{direct, via_cvt, connected}	relation-agnostic claim checking (Section 5.2.3)

5.2 The Graph Tool API

5.2.1 Rationale: Constrained Tools Instead of Free-Form Cypher

An obvious alternative is to let the model write Cypher directly. This was rejected. Generated-query approaches introduce a failure mode that is indistinguishable, in the results, from a reasoning failure: when a question is answered incorrectly, one cannot tell whether the agent reasoned badly or merely emitted invalid syntax. Since the thesis's central claims are about reasoning and verification, a confound that contaminates every error category is unacceptable.

The tools are therefore parameterised Cypher templates, written once and unit-tested (Section 7.10). They accept entity identifiers and return names alongside them, so the model always sees human-readable text while the program tracks identity. Every invocation is appended to a JSONL log with its arguments, result, and latency.

5.2.2 The Five Operations

Table 5.1 lists the operations. Note that there are five, not the four originally planned: verify_connection was added when the verification layer of Chapter 6 required a relation-agnostic adjacency test.

Three design decisions inside these operations are load-bearing.

Relations are returned with fanout counts, in both directions. The count is free signal: a relation with a fanout of forty thousand is almost never the edge that answers a specific question, and the scorer can use this without an extra query. Retrieving incoming as well as outgoing edges matters because Freebase's relation direction is a modelling artefact — a question about where someone was born may be answerable through people.person.place_of_birth outward or location.location.people_born_here inward.

Administrative predicates are filtered at the tool layer. Relations whose names begin with common., freebase., type., kg., user., dataworld., rdf-schema#, or owl# are dropped before the candidate list is built. These carry no factual content, and Section 5.10 shows what happens when they are not filtered: they consume beam slots and drag the agent into the schema rather than the data.

The relation list is capped at 300 entries, sorted by fanout. An earlier cap of 80 was raised after it was found to hide precisely the low-fanout, high-precision relations that answer specific questions — again, Section 5.10.

5.2.3 Mediator Traversal and Relation-Agnostic Adjacency

Section 4.3 established that 63.7% of the environment's nodes are unnamed mediators. If the agent saw these as candidate entities, most of its frontier would consist of nodes with no surface form and no meaning to a language model.

get_neighbors therefore hops through them transparently. When an expansion reaches a node flagged is_cvt, the tool immediately expands that node one further step, excluding the origin and excluding other mediators, and returns the entities on the far side. Each returned neighbour carries a via chain — one relation for a direct edge, two for a mediator-mediated one. The model reasons about logical neighbours; the log retains the full raw chain, so hop accounting stays exact.

verify_triple applies the same principle to checking: it matches *undirected* and accepts a mediator in the middle. Direction-strictness would generate false rejections, because the claims it checks come from generated text that has no notion of which way a Freebase edge points.

verify_connection goes further and drops the relation entirely, asking only whether two entities are adjacent directly or through one mediator. It exists because claim decomposition frequently produces a claim whose relation does not correspond to any single Freebase predicate — "X played for Y" may be realised through a roster mediator with no edge literally named for playing. Requiring an exact relation match in such cases rejects true claims; Section 6.3.1 explains where each of the two checks is used.

5.3 Agent State

The state is a typed dictionary threaded through every node. Table 5.2 groups its fields by purpose.

Two disciplines govern this structure and are worth stating because violating either produces bugs that are extremely hard to localise.

Nodes write, routers read. A router is a pure function that inspects state and returns an edge

Table 5.2: The agent state. Fields marked *append-only* accumulate across node executions through reducers; all others are overwritten by whichever node returns them.

Group	Fields
Question	qid, question, gold_q_entities
Plan	plan: ordered sub-objectives, each with a status (pending/active/done) and its resolved entities
Search	anchors (current frontier), traversed (*append-only* triple log), backtrack_stack, banned, last_expanded
Answer	candidate_answers, draft, answer, answer_entities, supporting_triples
Verification	supported_claims, unsupported_claims, unsupported_claim_objs
Control	budget (meter, mutated in place), trace (*append-only*), and the router scratch keys _eval_decision, _last_n_new, _frontier_max_score

label; it never mutates. Any signal a router needs must therefore be deposited in state by the node preceding it — which is what the three underscore-prefixed scratch keys are for. The explorer records how many triples it added and the best score it saw; the evaluator records its decision; the routers read those and decide.

The budget meter is mutated in place and never returned as a delta. It is a mutable object shared by reference across all nodes, because counters that are merged rather than mutated can be silently lost when two nodes return overlapping state. Enforcement must not be able to drift.

5.4 The Planner

5.4.1 Decomposition into Ordered Sub-Objectives

The planner makes one model call, converting the question into an ordered chain of sub-objectives plus the entity mentions to seed the search. The chain is strictly sequential; later objectives reference earlier results with a #N back-reference. A general dependency-graph planner was not built, because a sequential chain covers the composition structure that dominates CWQ and a DAG planner is a research problem in its own right.

The prompt is few-shot with one example per question archetype — single hop, composition, conjunction, superlative — and the single-hop example exists to demonstrate that *not* decomposing is a valid output. Over-decomposition of simple questions is a real failure mode, and Section 8.7 shows it is not hypothetical: on the shallower benchmark, removing the planner improves accuracy.

The prompt also instructs the model to extract only specific named entities as topic mentions, and explicitly not generic type words such as “celebrity”, “university”, “country”, or “senator”. That instruction was added in response to evidence discussed in Section 5.10: type words resolve

successfully to type nodes in the graph, consuming anchor slots with entities that lead nowhere.

5.4.2 Plan Validation and Degenerate-Plan Handling

The planner's output is validated, not trusted. Four checks run: the plan must contain a non-empty list of sub-objectives; it must contain no more than four; every #N reference must point strictly backwards; and topic mentions must be present. The back-reference check catches the planner's most common structural error, a step that references itself or a future step.

If any check fails — or if the model call itself raises — the planner falls back to a single sub-objective consisting of the whole question, seeded with the dataset's topic entities. This is a deliberate design principle: **planning can degrade but can never abort a run.** A degraded plan is a data point that appears in the results; a crash is a lost question that biases them.

Anchor seeding has two modes. In the default benchmark configuration, anchors are seeded from the dataset's annotated topic entities, which isolates reasoning from entity linking and is the standard protocol in this literature [7, 9]. The alternative seeds from the planner's own extracted mentions, giving the end-to-end number. Because Section 4.6 showed resolution succeeds on 99.86% of mentions, the two modes differ very little in this environment.

Setting the planner ablation flag bypasses the model call entirely and installs the fallback plan directly, which is what makes "no planner" a one-configuration-line change rather than a separate code path.

5.5 The Explorer

The explorer performs one expansion of the frontier. It is the only node that touches the graph in bulk, and its structure is the heart of the navigation policy.

5.5.1 Frontier Construction and the Single Scoring Pass

The node proceeds in five steps:

1. **Checkpoint.** The current frontier, the current depth, and the best score seen at this point are pushed onto the backtrack stack.

2. **Gather.** For every anchor, get_relations is called and the results are collected into a *single* candidate list, each row tagged with the anchor it came from.

3. **Score.** The whole list is scored in one pass (Section 5.5.2). Pooling candidates across anchors before scoring is what allows the hybrid scorer to consult the model *once* per expansion rather than once per anchor.

4. **Beam.** Candidates are taken in descending score order into per-anchor buckets of width three, skipping any (anchor, relation, direction) triple on the ban list (Section 5.6.3).

5. **Expand.** Each selected edge is expanded with get_neighbors; the resulting triples are appended to the traversal log, and the neighbours become frontier candidates.

The new frontier is deduplicated by entity identifier, sorted by the score of the edge that produced each entity, and truncated to the anchor cap. Sorting by score rather than by insertion order matters: an insertion-ordered frontier keeps whichever entities happened to be produced first, which causes the agent to drift away from the anchor that mattered.

5.5.2 The Hybrid Scoring Function

Each candidate edge receives a score blending semantic similarity with a model judgement:

$$\text{score}(c) \;=\; \alpha \cdot \cos\big(\mathbf{v}_{\text{rel}(c)},\, \mathbf{v}_{\text{obj}}\big) \;+\; (1-\alpha)\cdot \text{LLM}(c),$$

where $\mathbf{v}_{\text{rel}(c)}$ is the precomputed embedding of the candidate's relation name (Section 4.5), $\mathbf{v}_{\text{obj}}$ is the embedding of the *rendered active sub-objective*, and $\text{LLM}(c) \in [0,1]$ rates how likely following that edge from that anchor is to advance the sub-objective.

Three properties of this design deserve emphasis.

The objective, not the question, is embedded. Hop two of a two-hop question is scored against "find the director of Titanic", with the back-reference already substituted by the resolved entity name, rather than against the original question text. This is the concrete mechanism by which decomposition is supposed to help, and isolating it is what makes the planner ablation interpretable.

The model scores only a shortlist. Candidates are pre-ranked by embedding similarity and only the top twenty are shown to the model, in one batched call. Without the pre-filter, an expansion from five anchors with hundreds of relations each would be unaffordable.

The blend is applied after the call, not inside it. The scoring prompt contains no configuration value — no α, no τ. This is not cosmetic: because the response cache (Section 5.9) keys on prompt content, a prompt free of configuration means every condition in an α sweep shares the same cached model responses, and the sweep costs one set of calls instead of one per condition. Embedding a configuration value in that prompt would multiply sweep cost by the number of conditions without changing any result.

If the model call fails or the budget is exhausted, the scorer degrades silently to embedding-only scoring rather than aborting the expansion. Setting $\alpha = 1$ disables the model term entirely, which is the embedding-only ablation.

5.6 The Evaluator and Routing Policy

5.6.1 Sub-Objective Completion Judgment

The evaluator makes one model call per expansion. It is shown the active sub-objective, a window of retrieved facts, and the remaining budget, and returns a decision (continue, backtrack, or answer), whether the objective is complete, and which entities satisfy it.

The fact window is *relevance-ranked, not recent*. Traversed triples are verbalised, embedded, and the thirty most similar to the current objective are shown, in traversal order. The distinction is not incidental: a large expansion can add hundreds of triples, and a window showing the most recent thirty will push the answer out of view in exactly the situations where the expansion succeeded. Section 5.10 documents the case that forced this change.

5.6.2 The Groundedness Filter on Resolutions

The evaluator's resolved entity names are not accepted as given. Each is looked up against a table built from the tail entities of the *traversed* triples, and any name that does not appear there is discarded.

This filter is small in code and significant in effect. A language model asked which entities satisfy an objective will readily list entities it knows from training but never retrieved. The filter refuses to launder that parametric recall into the agent's evidence. Section 5.10 records a case in which the evaluator confidently resolved nine correct countries that appeared in no retrieved triple, and the filter rejected all nine — costing a point on that question while preserving the property that every entity the system proposes came from the graph.

Resolutions that survive the filter are accumulated into candidate_answers on *every* evaluation, whether or not the objective was judged complete. An earlier design discarded resolutions when the objective was incomplete, which threw away correct answers found mid-run.

When an objective completes and further objectives remain, the plan advances, the decision is forced to continue, and the resolved entities become the next frontier. When the last objective completes, the decision is forced to answer.

5.6.3 Backtracking Triggers and the Backtrack Stack

Backtracking fires on the first of three conditions, evaluated in this order:

1. **Dead end** — the last expansion produced no new triples.
2. **Low score** — the best candidate score at the frontier fell below the threshold τ.
3. **Evaluator veto** — the model judged the direction a dead end.

The order matters. The first two are deterministic and cost nothing; checking them before deferring to the model's judgement means a mechanically hopeless frontier is abandoned without an argument about it.

The backtracker does three things. It pops the *highest-scoring* snapshot from the stack — not the most recent one, which would make backtracking a simple undo. It restores that snapshot's anchors and depth. And, critically, it adds every (anchor, relation, direction) triple expanded since that snapshot to a **ban list** that the explorer consults on its next pass.

The ban list is what makes backtracking a search operation rather than a loop. Without it, restoring a frontier is pointless: a deterministic scorer facing an identical frontier selects identical edges, producing identical facts and an identical decision to backtrack, until the backtrack budget is exhausted. With it, a backtrack forces the scorer to its next-best alternatives, which is what "exploring an alternative branch" actually requires. Section 5.10 shows both behaviours in the logs.

5.6.4 Routing

route_after_eval is pure Python and enforces budgets before preferences. Wall-clock and depth are checked first; if either is exhausted the agent is routed to answer with whatever it has. If a backtrack trigger fired, the agent backtracks if the backtrack budget permits and otherwise answers. Only then is the evaluator's own decision honoured. Routers are deterministic by construction: no adversarial model output can change what they do.

5.7 The Answerer

The answerer produces the response from the traversed facts and the accumulated candidate entities. It has three paths. Normally it makes one model call. If the model budget is exhausted, it falls back without a call to the resolved entities of completed plan steps, or failing that to the accumulated candidates, producing a degraded but non-empty answer. If the call raises for any other reason, the error is recorded in the trace and the answer is an explicit failure string.

The distinction between the second and third paths is deliberate. An earlier version caught all exceptions as budget exhaustion, which silently misreported genuine errors as expected behaviour — exactly the kind of defect that inflates apparent robustness.

5.8 Budgets and Termination Guarantees

Budgets do three jobs: they bound cost, they guarantee termination, and they produce the efficiency data of Section 8.6. Table 5.3 lists them.

Table 5.3: Budget configuration. The configuration is hashed and the hash recorded in every run record, so results can always be attributed to the configuration that produced them.

Parameter	Value	Enforced in
max_depth	4	route_after_eval
beam_width	3	explorer (slice)
max_anchors	5	explorer (slice)
max_backtracks	3	route_after_eval
max_llm_calls	25	LLM client
max_verify_iters	2	route_after_verify
max_seconds	300	route_after_eval

Each budget has exactly one enforcement site, which is what keeps enforcement auditable. The model-call budget is checked inside the client before every call, so no node can bypass it; exceeding it raises an exception that each node catches into a degraded path rather than a crash. Depth, backtracks, and wall-clock are checked only in the post-evaluation router. Verification iterations are checked only in the post-verification router. Beam width and anchor cap are not checks at all but slices inside the explorer.

Together these give a termination guarantee that does not depend on model behaviour: every cycle in Figure 5.1 passes through a router that decrements or checks a monotone counter, so no sequence of model outputs, however adversarial, can produce a non-terminating run. This property is verified directly by the mechanics tests of Section 7.10, which script an evaluator that always says continue and assert that the depth budget halts it.

5.9 Instrumentation and Reproducibility

Two logs are written per run. The *tool log* records every graph operation with its arguments, result size, and latency. The *run log* records, per question: the answer and its extracted entities, the draft, the verification outcome and any unsupported claims, the count of supporting triples, every backtrack with its trigger reason and restored depth, the full decision trace, the budget snapshot, wall-clock time, and — for provenance — the backbone description, the budget configuration hash, the run configuration, and the current git commit.

The response cache deserves separate description because it underwrites the thesis's reproducibility claims. Every model call is keyed by a hash over the model identifier, temperature, reasoning-effort setting, and the full prompt text; hits are served from disk. The subtle part is that a cache hit must be *indistinguishable* from a live call inside the agent: the budget check still runs, the call counter still increments, and the original token counts are replayed into the meter from the stored record. A fully cached rerun therefore reproduces the original run's budget snapshots exactly, rather than reporting zero tokens. A separate counter records cache hits so

that live and replayed runs can still be told apart in analysis.

This has three consequences. Repeated experiments cost nothing. A code change that is not supposed to alter behaviour can be verified by confirming that a rerun is served entirely from cache. And because the key includes the dated model snapshot, a silent server-side change to a model alias cannot poison previously cached results.

5.10 Design Validation on the Development Set

The first end-to-end run of the assembled framework, over a twenty-question development set drawn from the training splits, answered roughly three to four questions correctly. The final configuration answers thirteen to fourteen. The interval between those numbers is not incidental tuning; it is where several of the mechanisms described above were shown to be necessary, and it is documented here because each fix is evidence that a component earns its place.

The development set was constructed to exercise specific machinery rather than to sample the benchmark distribution: six single-hop questions, eight two-hop compositions, three conjunctions, two whose shortest path traverses a mediator, and one question about a deliberately non-existent entity whose correct answer is a refusal. Questions were drawn from the training and validation splits only, never from test, so that everything tuned against them remains outside the evaluation.

The first run produced no crashes, terminated cleanly on every question, and abstained rather than confabulated in almost every failure — which was the actual exit criterion for the framework's mechanics. But its thirteen failures collapsed into five systematic causes, summarised in Table 5.4.

Three observations from this exercise are worth carrying forward, because they bear directly on later chapters.

The two confabulations in the first run were caused by a retrieval defect, not a generation defect. Two questions about national currencies produced a confident "United States Dollar". The relation cap had hidden the currency_used relation; the beam surfaced triples about GNI per capita denominated in dollars; and the answerer pattern-matched. No retrieved triple supported the answer. After the cap was raised both questions resolved correctly — and at roughly a third of the token cost. These two traces are the motivating example for Chapter 6: a claim-level check against the traversed triples would have caught them regardless of the retrieval defect, which is precisely the argument for verifying before emitting.

The groundedness filter on resolutions was validated by a failure. On one question the evaluator listed nine correct countries; none appeared in any retrieved triple, because the beam had gone down an unrelated branch. The filter rejected all nine, and the system abstained. This cost a point. It also demonstrated, one layer earlier than intended, that a language model inside

Table 5.4: Root causes of failure in the first end-to-end run, and the mechanism each one motivated. Every fix is described in the section indicated.

Observed failure	Mechanism introduced	Section
Backtracking was a deterministic no-op: the restored frontier produced identical scores, facts, and decisions until the budget drained	Ban list on abandoned (anchor, relation, direction) triples	Section 5.6.3
The evaluator's fact window showed the most recent thirty triples, so a large successful expansion pushed the answer out of view	Relevance-ranked fact window	Section 5.6
The relation list, capped at 80 by descending fanout, hid low-fanout precise relations behind hub relations	Cap raised to 300, plus the administrative-predicate blocklist	Section 5.2
Entities resolved while an objective was still incomplete were discarded	candidate_answers accumulated on every evaluation	Section 5.6
The frontier was truncated in insertion order, so the agent drifted away from the anchor that mattered	Frontier sorted by edge score before truncation	Section 5.5

an agentic scaffold will assert entities it never retrieved — and that catching this requires an explicit check, because nothing about the model's confidence distinguishes the nine countries it recalled from entities it actually found.

One instrumentation defect was found and removed. A counter reporting how many banned expansions had been skipped was computing the number of anchors instead. It was reporting a plausible-looking number that was unrelated to what it claimed to measure. It was corrected rather than deleted, but the episode motivates a standing rule adopted for the remainder of the work: a metric that has never been checked against a case where its correct value is known independently should not be trusted, and should not be analysed.

5.11 The Complete Navigation Algorithm

Algorithm 1 states the loop.

Algorithm 1 AGR navigation (verification layer abstracted; see Chapter 6)

Require: question q, graph G, budgets B, blend α, threshold τ
Ensure: answer A with supporting triples S

$P \leftarrow \textsc{Plan}(q)$; **if** invalid **then** $P \leftarrow [\,q\,]$
$F \leftarrow \textsc{SeedAnchors}(P)$; $T \leftarrow \emptyset$; $C \leftarrow \emptyset$; $stack \leftarrow \emptyset$; $ban \leftarrow \emptyset$
while budgets permit **do**
  push $(F, depth, score)$ onto $stack$
  $R \leftarrow \bigcup_{e \in F} \textsc{GetRelations}(e)$
  $scored \leftarrow \alpha \cos(\cdot) + (1-\alpha)\mathrm{LLM}(\cdot)$ over R against the active objective
  $E \leftarrow$ top-β of $scored$ per anchor, excluding ban
  $T \leftarrow T \cup \textsc{GetNeighbors}(E)$; $F \leftarrow$ top-m new entities by score
  $(d, done, \rho) \leftarrow \textsc{Evaluate}(\text{objective}, \textsc{TopFacts}(T))$
  $\rho \leftarrow \{x \in \rho : x$ appears in $T\}$; $C \leftarrow C \cup \rho$ {groundedness filter}
  if $done$ **then**
    advance P; **if** no objective remains **then** $d \leftarrow$ answer **else** $F \leftarrow \rho$
  end if
  if $|T_{\text{new}}| = 0$ **or** $score < \tau$ **or** $d =$ backtrack **then**
    $ban \leftarrow ban \cup E$; $(F, depth) \leftarrow$ pop highest-scoring snapshot
  else if $d =$ answer **then**
    break
  end if
end while
$(A, S) \leftarrow \textsc{VerifyAndAnswer}(q, T, C)$ {Chapter 6}
return (A, S)

Chapter 6

The Structural Verification Layer

6.1 Motivation: Verifying Before Emitting, Not After

6.2 Claim Decomposition of the Draft Answer

6.3 Two-Stage Claim Checking

6.3.1 The Structural Check

6.3.2 The Entailment Check

6.3.3 Why the Structural Check Runs First

6.4 Repair Policies for Unsupported Claims

6.4.1 Targeted Re-Exploration Under Remaining Budget

6.4.2 Answer Rewriting and Hedging

6.4.3 Iteration Cap and the Draft-Only Fallback

6.5 Entity Filtering of the Final Answer

6.6 Output Contract: Answer Paired with Supporting Triples

6.7 A Worked Example

6.8 Failure Modes by Construction: Wrongful Rejection and Wrongful Acceptance

Chapter 7

Experimental Setup

7.1 Mapping Experiments to Research Questions

7.1.1 Pre-Registration and the No-Tuning Policy

7.1.2 Metrics Proposed but Not Run

7.2 Test Sets

7.2.1 WebQSP

7.2.2 ComplexWebQuestions

7.2.3 Stratified Sampling, Seeds, and Published Question IDs

7.2.4 Hop-Count Stratification

7.3 Backbone Model Selection and Qualification

7.3.1 Trajectory Stability Under Temperature-Zero Decoding

7.3.2 Response Caching as the Reproducibility Backstop

7.4 Baseline Systems

7.4.1 No-Retrieval LLM (Parametric-Memory Control)

7.4.2 Vector-RAG over Verbalized Triples

7.4.3 Static GraphRAG

7.4.4 Think-on-Graph (Agentic Baseline)

Chapter 8

Results

8.1 Development-Set Tuning Outcomes

8.2 Main Results on WebQSP

8.3 Main Results on ComplexWebQuestions

8.4 Accuracy Broken Down by Hop Count

8.5 Groundedness and Hallucination Rate Across Systems

8.6 Efficiency and Cost

8.6.1 Token and Call Budgets per System

8.6.2 The Accuracy–Cost Frontier

8.7 Ablation Study

8.7.1 Without the Planner: A Stratum-Dependent Effect

8.7.2 Without Backtracking

8.7.3 Without the Verification Layer

8.7.4 Embedding-Only Scoring

8.7.5 Paired Significance Testing and Statistical Power

8.8 Findings Against RQ1, RQ2, and RQ3

Chapter 9

Error Analysis and Discussion

9.1 Annotation Protocol and the Failure Taxonomy

9.1.1 Category and Subtype Schema

9.1.2 Sampling Asymmetry: Census versus Stratified Sample

9.2 Distribution of Failure Categories Across Datasets

9.3 Decomposition, Drafting, and Answer-Selection Errors

9.3.1 Right Candidate Retrieved, Wrong One Drafted

9.4 Relation-Selection and Navigation Errors

9.5 Verifier Rejection and Acceptance Errors

9.5.1 Defining the Error Polarities

9.5.2 Wrongly Rejected: Semantically Correct, Structurally Unmatched

9.5.3 Wrongly Accepted: Unsupported Claims Passed as Grounded

9.5.4 Rate for One Polarity, and a Blank for the Other

9.6 Knowledge-Graph Gaps: Literals, Temporal Qualifiers, and Ordinals

9.7 The Echo Attractor: Answering with the Topic or an

Chapter 10

Conclusion

10.1 Summary of Contributions

10.2 Limitations

10.3 Future Work

10.3.1 Path Fidelity Against Gold SPARQL Relation Chains

10.3.2 Logging Accepted Claims to Make Wrongful Acceptance Measurable

10.3.3 LLM-Constructed Knowledge Graphs as a Controlled Comparison

10.3.4 Rollout-Based Value Estimation

10.3.5 Temporal and Multi-Source Knowledge Environments

10.3.6 Transfer to High-Stakes Domains

References

[1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, pp. 1–55, 2025.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[3] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, and J. X. Yu, "A survey of graph meets large language model: Progress and future directions," *arXiv preprint arXiv:2311.12399*, 2023.

[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[5] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems*, vol. 37, pp. 59532–59569, 2024.

[6] D. Li, Y. Niu, Y. Ai, X. Zou, B. Qi, and J. Liu, "T-GRAG: A dynamic GraphRAG framework for resolving temporal conflicts and redundancy in knowledge retrieval," in *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 11880–11889, 2025.

[7] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, and J. Guo, "Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.07697.

[8] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, and H. Xiong, "Plan-on-graph: Self-correcting adaptive planning of large language model on knowledge graphs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. arXiv:2410.23875.

[9] L. Luo, Y.-F. Li, G. Haffari, and S. Pan, "Reasoning on graphs: Faithful and interpretable large language model reasoning," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01061.

[10] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, and J.-R. Wen, "KG-Agent: An efficient autonomous agent framework for complex reasoning over knowledge graph," *arXiv preprint arXiv:2402.11163*, 2024.

[11] Y. Xu, S. He, J. Chen, Z. Wang, Y. Song, H. Tong, K. Liu, and J. Zhao, "Generate-on-graph: Treat LLM as both agent and KG for incomplete knowledge graph question answering," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. arXiv:2404.14741.

[12] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, and J.-R. Wen, "StructGPT: A general framework for large language models to reason over structured data," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.09645.

[13] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.

[14] C. Ma, Y. Chen, T. Wu, A. Khan, and H. Wang, "Unifying large language models and knowledge graphs for question answering: Recent advances and opportunities," in *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*, pp. 1174–1177, 2025.

[15] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3563–3578, 2024.

[16] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[17] Y. Hu, W. Xuan, Q. Zhou, Z. Li, Y. Li, J. Hu, and F. Fang, "A self-correcting agentic graph RAG for clinical decision support in hepatology," *Frontiers in Medicine*, vol. 12, p. 1716327, 2025.

[18] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: A collaboratively created graph database for structuring human knowledge," in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pp. 1247–1250, 2008.

[19] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh, "The value of semantic parse labeling for knowledge base question answering," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 201–206, 2016.

[20] A. Talmor and J. Berant, "The web as a knowledge-base for answering complex questions," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 641–651, 2018.

[21] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*. O'Reilly Media, 2nd ed., 2015.

[22] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor, "Cypher: An evolving query language for property graphs," in *Proceedings of the 2018 International Conference on Management of Data*, pp. 1433–1445, 2018.

[23] J. Berant, A. Chou, R. Frostig, and P. Liang, "Semantic parsing on Freebase from question-answer pairs," in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pp. 1533–1544, 2013.

[24] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.

[26] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[28] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781, 2020.

[29] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992, 2019.

[30] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[31] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

[32] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *Proceedings of the 17th European Conference on Machine Learning*, pp. 282–293, 2006.

[33] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of monte carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.

[34] T. Shen, J. Wang, X. Zhang, and E. Cambria, "Reasoning with trees: Faithful question answering over knowledge graph," in *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 3138–3157, 2025.

[35] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[36] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[37] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[38] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[39] X. Tan, X. Wang, Q. Liu, X. Xu, X. Yuan, and W. Zhang, "Paths-over-graph: Knowledge graph empowered large language model reasoning," *arXiv preprint arXiv:2410.14211*, 2024.

[40] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR)*, 2024.

[41] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[42] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12076–12100, 2023.

[43] F. F. Bayat, L. Zhang, S. Munir, and L. Wang, "FactBench: A dynamic benchmark for in-the-wild language model factuality evaluation," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33090–33110, 2025.

[44] Y. Gu, S. Kase, M. Vanni, B. Sadler, P. Liang, X. Yan, and Y. Su, "Beyond I.I.D.: Three levels of generalization for question answering on knowledge bases," in *Proceedings of the Web Conference 2021*, pp. 3477–3488, 2021.

Index

Appendix A

Prompt Templates

Appendix B

Tool API Specification and Cypher Templates

Appendix C

Sampled Question IDs and Run Manifests

Appendix D

Annotation Instructions and Label Schema

Appendix E

Implementation Notes

Generated using Postgraduate Thesis LaTeX Template, Version 1.03. Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh.

This thesis was generated on Wednesday 5th August, 2026 at 9:42pm.